

Лабораторна робота №3

REST API з Express.js та інтеграційне тестування

Мета роботи

Метою лабораторної роботи є набуття практичних навичок розробки RESTful API на Express.js з TypeScript та написання інтеграційних тестів з використанням Supertest, зокрема:

- ініціалізація Express-проєкту з TypeScript та налаштування необхідних залежностей;
- визначення типів сутності через Zod-схеми та z.infer;
- реалізація in-memory сховища з використанням Map;
- побудова CRUD-маршрутів з правильними HTTP-статус-кодами;
- розділення конфігурації застосунку (app.ts) та запуску сервера (server.ts);
- написання інтеграційних тестів для всіх маршрутів з використанням Supertest.

Хід роботи

Лістинг:

app.ts:

```
import express from 'express';
import cors from 'cors';
import carRoutes from './routes/car';
import { errorHandler } from './middleware/errorHandler';

const app = express();

app.use(cors());
app.use(express.json());

app.use('/api/cars', carRoutes);

app.use(errorHandler);

export default app;
```

server.ts:

```
import app from './app';

const PORT = 3000;

app.listen(PORT, () => {
  console.log(`Сервер запущено на http://localhost:${PORT}`);
});
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Рак І. С.			Звіт з лабораторної роботи №3	Лім.	Арк.
Перевір.		Фуріхата Д. В.					Аркушів
Реценз.							1
Н. Контр.						ФІКТ, гр. ІПЗ-23-2	
Зав.каф.		Вакалюк Т.А.					
							10

errorHandler.ts:

```
import { Request, Response, NextFunction } from 'express';
import { ZodError } from 'zod';

export const errorHandler = (err: Error, req: Request, res: Response, next: NextFunction) => {
  if (err instanceof ZodError) {
    return res.status(400).json({
      message: 'Помилка валідації даних',
      errors: err.issues
    });
  }

  console.error(err);
  return res.status(500).json({ message: 'Внутрішня помилка сервера' });
};
```

validate.ts:

```
import { Request, Response, NextFunction } from 'express';
import { ZodSchema } from 'zod';

export const validate = (schema: ZodSchema) => {
  return (req: Request, res: Response, next: NextFunction) => {
    try {
      req.body = schema.parse(req.body);
      next();
    } catch (error) {
      next(error);
    }
  };
};
```

car.ts:

```
import { Router, Request, Response } from 'express';
import { carStorage } from '../storage/car';
import { createCarSchema, updateCarSchema } from '../schemas/car.schema';
import { validate } from '../middleware/validate';

const router = Router();

router.get('/available', (req: Request, res: Response) => {
  const cars = carStorage.getAll().filter(car => car.isAvailable);
  res.status(200).json(cars);
});

// GET. отримання всіх записів
router.get('/', (req: Request, res: Response) => {
  const { minYear, transmission } = req.query;

  const filter: any = {};
  if (minYear) filter.minYear = parseInt(minYear as string, 10);
  if (transmission) filter.transmission = transmission as string;

  const cars = carStorage.getAll(filter);
  res.status(200).json(cars);
});

// GET. отримання за ID
router.get('/:id', (req: Request<{ id: string }>, res: Response) => {
  const car = carStorage.getById(req.params.id);
  if (!car) {
    return res.status(404).json({ message: 'Автомобіль не знайдено' });
  }
});
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3	Арк.
						2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    res.status(200).json(car);
  });

// POST. створення нового автомобіля
router.post('/', validate(createCarSchema), (req: Request, res: Response) => {
  const newCar = carStorage.create(req.body);
  res.status(201).json(newCar);
});

// PATCH. Оновлення вже існуючого автомобіля
router.patch('/:id', validate(updateCarSchema), (req: Request<{ id: string }>, res: Response) => {
  const updatedCar = carStorage.update(req.params.id, req.body);
  if (!updatedCar) {
    return res.status(404).json({ message: 'Автомобіль не знайдено' });
  }
  res.status(200).json(updatedCar);
});

// DELETE. Видалення автомобіля
router.delete('/:id', (req: Request<{ id: string }>, res: Response) => {
  const isDeleted = carStorage.delete(req.params.id);
  if (!isDeleted) {
    return res.status(404).json({ message: 'Автомобіль не знайдено' });
  }
  res.status(204).send();
});

export default router;

```

car.schema.ts:

```

import { z } from 'zod';

export const createCarSchema = z.object({
  model: z.string().min(1, "Модель обов'язкова").max(100),

  description: z.string().max(500).optional(),

  year: z.number().int().min(1886).max(new Date().getFullYear()),
  transmission: z.enum(['manual', 'automatic', 'robotic']),
  isAvailable: z.boolean().default(true)
});

export const updateCarSchema = createCarSchema.partial();

export type CreateCarInput = z.infer<typeof createCarSchema>;
export type UpdateCarInput = z.infer<typeof updateCarSchema>;

export type CarEntity = CreateCarInput & {
  id: string;
  createdAt: Date;
  updatedAt: Date;
};

```

car.ts:

```

import { CarEntity, CreateCarInput, UpdateCarInput } from '../schemas/car.schema';
import crypto from 'crypto';

export interface CarFilter {
  minYear?: number;
  transmission?: 'manual' | 'automatic' | 'robotic';
}

class CarStorage {

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

private cars: Map<string, CarEntity> = new Map();

getAll(filter?: CarFilter): CarEntity[] {
    let result = Array.from(this.cars.values());

    if (filter) {
        if (filter.minYear) {
            result = result.filter(car => car.year >= filter.minYear!);
        }
        if (filter.transmission) {
            result = result.filter(car => car.transmission ===
filter.transmission);
        }
    }
    return result;
}

getId(id: string): CarEntity | undefined {
    return this.cars.get(id);
}

create(data: CreateCarInput): CarEntity {
    const id = crypto.randomUUID();
    const now = new Date();

    const newCar: CarEntity = {
        ...data,
        id,
        createdAt: now,
        updatedAt: now,
        isAvailable: data.isAvailable
    };

    this.cars.set(id, newCar);
    return newCar;
}

update(id: string, data: UpdateCarInput): CarEntity | null {
    const existingCar = this.cars.get(id);
    if (!existingCar) return null;

    const updatedCar: CarEntity = {
        ...existingCar,
        ...data,
        updatedAt: new Date()
    };

    this.cars.set(id, updatedCar);
    return updatedCar;
}

delete(id: string): boolean {
    return this.cars.delete(id);
}

reset(): void {
    this.cars.clear();
}
}

export const carStorage = new CarStorage();

```

car.test.ts:

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

import request from 'supertest';
import app from '../src/app';
import { carStorage } from '../src/storage/car';
import { CreateCarInput } from '../src/schemas/car.schema';

beforeEach(() => {
  carStorage.reset();
});

describe('Cars REST API', () => {

  const validCar: CreateCarInput = {
    model: 'Toyota Camry',
    description: 'Надійний седан',
    year: 2022,
    transmission: 'automatic',
    isAvailable: true
  };

  describe('POST /api/cars', () => {
    it('повинен створити новий автомобіль та повернути 201', async () => {
      const res = await request(app)
        .post('/api/cars')
        .send(validCar)
        .expect(201);

      expect(res.body).toHaveProperty('id');
      expect(res.body.model).toBe(validCar.model);
      expect(res.body.isAvailable).toBe(true);
      expect(res.body).toHaveProperty('createdAt');
    });

    it('повинен повернути 400, якщо відсутні обов\'язкові поля', async () => {
      const invalidCar = { year: 2020 };

      const res = await request(app)
        .post('/api/cars')
        .send(invalidCar)
        .expect(400);

      expect(res.body.message).toBe('Помилка валідації даних');
      expect(res.body.errors.length).toBeGreaterThan(0);
    });

    it('повинен повернути 400, якщо передано невалідне значення enum', async
  () => {
    const invalidCar = { ...validCar, transmission: 'magic' };

    await request(app)
      .post('/api/cars')
      .send(invalidCar)
      .expect(400);
  });

  });

  describe('GET /api/cars', () => {
    it('повинен повернути порожній масив, якщо немає записів', async () => {
      const res = await request(app).get('/api/cars').expect(200);
      expect(res.body).toEqual([]);
    });

    it('повинен повернути масив усіх збережених автомобілів', async () => {
      carStorage.create(validCar);
      carStorage.create({ ...validCar, model: 'Honda Civic' });
    });
  });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        const res = await request(app).get('/api/cars').expect(200);
        expect(res.body).toHaveLength(2);
    });
});

describe('GET /api/cars/:id', () => {
    it('повинен повернути автомобіль за існуючим ID', async () => {
        const car = carStorage.create(validCar);

        const res = await request(app)
            .get(`/api/cars/${car.id}`)
            .expect(200);

        expect(res.body.id).toBe(car.id);
        expect(res.body.model).toBe(car.model);
    });

    it('повинен повернути 404, якщо автомобіль не знайдено', async () => {
        await request(app)
            .get('/api/cars/non-existent-id')
            .expect(404);
    });
});

describe('PATCH /api/cars/:id', () => {
    it('повинен частково оновити автомобіль та повернути 200', async () => {
        const car = carStorage.create(validCar);

        const res = await request(app)
            .patch(`/api/cars/${car.id}`)
            .send({ year: 2025, isAvailable: false })
            .expect(200);

        expect(res.body.year).toBe(2025);
        expect(res.body.isAvailable).toBe(false);
        expect(res.body.model).toBe(validCar.model);
    });

    it('повинен повернути 400 при спробі встановити невалідні дані', async () => {
        const car = carStorage.create(validCar);

        await request(app)
            .patch(`/api/cars/${car.id}`)
            .send({ year: 1500 })
            .expect(400);
    });

    it('повинен повернути 404 при оновленні неіснуючого ID', async () => {
        await request(app)
            .patch('/api/cars/fake-id')
            .send({ year: 2023 })
            .expect(404);
    });
});

describe('DELETE /api/cars/:id', () => {
    it('повинен видалити автомобіль та повернути 204', async () => {
        const car = carStorage.create(validCar);

        await request(app)
            .delete(`/api/cars/${car.id}`)
            .expect(204);
    });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        expect(carStorage.getById(car.id)).toBeUndefined();
    });

    it('повинен повернути 404 при спробі видалити неіснуючий автомобіль',
    async () => {
        await request(app)
            .delete('/api/cars/fake-id')
            .expect(404);
    });
});

describe('GET /api/cars (Query Filters)', () => {
    beforeEach(() => {
        carStorage.create({ ...validCar, model: 'Old Manual', year: 2005,
        transmission: 'manual', isAvailable: true });
        carStorage.create({ ...validCar, model: 'New Auto', year: 2024,
        transmission: 'automatic', isAvailable: false });
        carStorage.create({ ...validCar, model: 'Mid Robotic', year: 2015,
        transmission: 'robotic', isAvailable: true });
    });

    it('повинен фільтрувати за роком (minYear)', async () => {
        const res = await
        request(app).get('/api/cars?minYear=2020').expect(200);
        expect(res.body).toHaveLength(1);
        expect(res.body[0].model).toBe('New Auto');
    });

    it('повинен фільтрувати за трансмісією', async () => {
        const res = await
        request(app).get('/api/cars?transmission=manual').expect(200);
        expect(res.body).toHaveLength(1);
        expect(res.body[0].model).toBe('Old Manual');
    });

    it('повинен комбінувати фільтри', async () => {
        const res = await
        request(app).get('/api/cars?minYear=2020&transmission=automatic').expect(200);
        expect(res.body).toHaveLength(1);
        expect(res.body[0].model).toBe('New Auto');

        const emptyRes = await
        request(app).get('/api/cars?minYear=2020&transmission=manual').expect(200);
        expect(emptyRes.body).toHaveLength(0);
    });
});

describe('GET /api/cars/available (Специфічний маршрут)', () => {
    it('повинен повертати лише доступні автомобілі', async () => {
        carStorage.create({ ...validCar, model: 'Car 1', isAvailable: true });
        carStorage.create({ ...validCar, model: 'Car 2', isAvailable: false });

        carStorage.create({ ...validCar, model: 'Car 3', isAvailable: true });

        const res = await request(app).get('/api/cars/available').expect(200);

        expect(res.body).toHaveLength(2);
        res.body.forEach((car: any) => {
            expect(car.isAvailable).toBe(true);
        });
    });
});

describe('Обробник помилок (500)', () => {

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        it('повинен повернути статус 500, якщо сталася внутрішня помилка сервера',
async () => {
    jest.spyOn(carStorage, 'getAll').mockImplementationOnce(() => {
        throw new Error('Системний збій');
    });

    const res = await request(app)
        .get('/api/cars')
        .expect(500);

    expect(res.body.message).toBe('Внутрішня помилка сервера');
});
});
});
});

```

PASS tests/car.test.ts

Cars REST API

POST /api/cars

- ✓ повинен створити новий автомобіль та повернути 201 (50 ms)
- ✓ повинен повернути 400, якщо відсутні обов'язкові поля (5 ms)
- ✓ повинен повернути 400, якщо передано невалідне значення enum (3 ms)

GET /api/cars

- ✓ повинен повернути порожній масив, якщо немає записів (5 ms)
- ✓ повинен повернути масив усіх збережених автомобілів (3 ms)

GET /api/cars/:id

- ✓ повинен повернути автомобіль за існуючим ID (4 ms)
- ✓ повинен повернути 404, якщо автомобіль не знайдено (5 ms)

PATCH /api/cars/:id

- ✓ повинен частково оновити автомобіль та повернути 200 (4 ms)
- ✓ повинен повернути 400 при спробі встановити невалідні дані (4 ms)
- ✓ повинен повернути 404 при оновленні неіснуючого ID (4 ms)

DELETE /api/cars/:id

- ✓ повинен видалити автомобіль та повернути 204 (4 ms)
- ✓ повинен повернути 404 при спробі видалити неіснуючий автомобіль (3 ms)

GET /api/cars (Query Filters)

- ✓ повинен фільтрувати за роком (minYear) (4 ms)
- ✓ повинен фільтрувати за трансмісією (5 ms)
- ✓ повинен комбінувати фільтри (7 ms)

GET /api/cars/available (Специфічний маршрут)

- ✓ повинен повертати лише доступні автомобілі (3 ms)

Обробник помилок (500)

- ✓ повинен повернути статус 500, якщо сталася внутрішня помилка сервера (28 ms)

Рис.1 Зображення результату

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
src	100	100	100	100	
app.ts	100	100	100	100	
src/middleware	100	100	100	100	
errorHandler.ts	100	100	100	100	
validate.ts	100	100	100	100	
src/routes	100	100	100	100	
car.ts	100	100	100	100	
src/schemas	100	100	100	100	
car.schema.ts	100	100	100	100	
src/storage	100	100	100	100	
car.ts	100	100	100	100	
Test Suites: 1 passed, 1 total					
Tests: 17 passed, 17 total					
Snapshots: 0 total					
Time: 1.426 s, estimated 3 s					
Ran all test suites.					

Рис.2 Зображення результату

Висновок: У ході виконання лабораторної роботи №3 було набуто практичних навичок розробки RESTful API на базі Express.js з використанням TypeScript та інтеграційного тестування за допомогою фреймворку Supertest .

Основними результатами роботи є:

- **Архітектура та налаштування:** Успішно ініціалізовано Express-проект, налаштовано залежності та реалізовано правильне розділення конфігурації застосунку (app.ts) від логіки запуску сервера (server.ts), що є критично важливим для коректного тестування .
- **Типізація та валідація:** Забезпечено сувору перевірку вхідних даних за допомогою бібліотеки Zod. Типи TypeScript (CreateCarInput, UpdateCarInput тощо) було автоматично виведено зі схем валідації через z.infer, що усунуло необхідність дублювання коду та ручних перевірок .
- **Зберігання даних:** Реалізовано in-memory сховище на базі об'єкта Map для сутності "Автомобіль". Сховище підтримує всі CRUD-операції та містить метод скидання стану для ізоляції тестового середовища .
- **Маршрутизація та обробка помилок:** Побудовано повноцінний набір CRUD-маршрутів з дотриманням семантики REST (використання

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		9

правильних HTTP-методів та статус-кодів 200, 201, 204, 400, 404, 500) .

Впроваджено middleware для автоматичної валідації запитів та централізований errorHandler для розмежування помилок валідації та внутрішніх збоїв сервера .

- **Фільтрація даних:** Згідно з принципом єдиної відповідальності, логіку фільтрації за query-параметрами (minYear, transmission) було реалізовано безпосередньо на рівні сховища. Також додано специфічний маршрут для отримання лише доступних автомобілів .
- **Інтеграційне тестування:** Написано комплексний набір із 17 інтеграційних тестів з використанням Jest та Supertest. Тестами повністю покрито всі маршрути, включаючи перевірку успішних сценаріїв, обробку невалідних даних, відсутніх ресурсів та фільтрацію . Досягнуто 100% покриття коду (code coverage).

<https://git.ztu.edu.ua/node-js/lab3>

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.23.000 – Лр.3	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		10